МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хиймэл оюуны үндэс

Лекц № 3

This e-learning content was developed by the support of KOICA

МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хичээл 3.1
Хиймэл оюун ухааны хэрэгслүүд

Компьютерын ухааны салбар

This e-learning content was developed by the support of KOICA

Энэ удаагийн лекцээр

- Хиймэл оюун ухаанд өргөн хэрэглэгддэг нийлмэл өгөгдлийн төрлүүд болон нэрлэсэн, нэрлээгүй функцуудын ажиллах зарчим, хэрэглээ зэргийг судлах болно. Үүнд:
- Массив ба жагсаалт;
- Tuple, Толь бичиг, олонлог;
- Функц: Lamda, Filter, Map, Reduce;
- Дүгнэлт.

Ж.Алимаа. ШУТИС, МХТС, Компьютерийн ухааны салбар, дэд профессор (PhDДоктор). (e-mail: alimaaj@must.edu.mn)



Суралцахуйн үр дүн

Хиймэл оюун ухааны хэрэгслүүдийн нэг болох Python программчлалын хэлний дараахь ойлголтуудыг судалж тэдгээрийг практикт хэрэглэх жишээтэй танилцах болно. Үүнд:

- Массив ба жагсаалтын талаарх ойлголттой танилцах, хэрэглэх;
- Tuple, Толь бичиг, олонлогийн онцлогийг судлах ба хэрэглэх;
- Функц: Lamda, Filter, Map, Reduce-н ажиллах болон ашиглах зарчим, хэрэглэх.

Массив

Array- массив нь ижил төрлийн өгөгдлүүдийг агуулсан нийлмэл өгөгдлийн төрөл юм.

Python хэлэнд ашиглах бичиглэлийн хэлбэр :

```
import array as arr

# массив үүсгэх
a = arr.array('i', [1, 2, 3])     /

# массивын элементүүдийг хэвлэх
for i in range(0, 3):
    print(a[i], end=" ")
```

Үр дүн: 1, 2, 3

Жагсаалт

List-жагсаалт нь индексээр хандах боломжтой өгөгдлийн дараалал юм.

Жагсаалтыг **[ ]** хаалтаар дүрслэх бөгөөд жагсаалтын өгөгдлүүд нь хэрэглээний хувьд ихэвчлэн нэгэн төрлийн өгөгдлүүд байдаг гэхдээ, ялгаатай өгөгдлийн төрлүүд байж болно.

Жагсаалтын элементүүдийг өөрчлөх боломжтой. Жагсаалт дээр гүйцэтгэх боломжтой үйлдлүүдийг хүснэгтэнд харуулав.

Функц	Тайлбар	Жишээ	Үр дүн
“+” үйлдэл	Жагсаалтуудыг залгах	List1=[10,20,30] list2=[40,60,80] List3=list1+list2	List3 [10,20,30,40,60, 80]
del(жагсаалтын нэр[индекс])	Тодорхой индекстэй элементийг жагсаалтаас устгах	del(List3[2])	[10,20,40,60,80]
жагсаалтын нэр.pop()	Жагсаалтын сүүлийн элементийг устгах, устгаж буй утгыг буцаах	list=[10,20,30] list.pop(1)	20
List(тэмдэгт мөрийн хувьсагч)	Тэмдэгт мөрийг жагсаалт руу хувиргах	Str=”Python” List(Str)	[‘P’, ‘y’, ‘t’, ‘h’, ‘o’, ‘n’]
append()	Жагсаалтын төгсгөлд элемент нэмнэ	li=["ааруул","ээзгий"] li.append("Өрөм") li2=["сүү","тараг"] li.append(li2) print(li)	['ааруул', 'ээзгий', 'Өрөм', ['сүү', 'тараг']]

Жагсаалт

Функц	Тайлбар	Жишээ	Үр дүн
clear()	Жагсаалтын бүх элементийг устгана	jims = ['алим', 'банан', 'тоор', 'чангааНз'] jims.clear() print(jims)	[]
copy()	Жагсаалтын хуулбарыг буцаана	jims = ['алим', 'банан', 'тоор', 'чангааНз'] x=jims.copy() print('x=',x)	x= ['алим', 'банан', 'тоор', 'чангааНз']
count()	Заасан утгатай элемент жагсаалтанд хэдэн удаа давтагдан орсон тоог буцаана	jims = ['алим', 'банан', 'тоор', 'чангааНз','чангааНз'] x=jims.count("чангааНз") print('x=',x)	x=2
extend()	Одоогийн жагсаалтын төгсгөлд элементүүд нэмнэ (давтагдах элемент байж болно)	jims = ['алим','тоор','чангааНз'] name=['Туяа','Болор','Болд'] jims.extend(name) print('Result:',jims)	Result: ['алим', 'тоор', 'чангааНз', 'Туяа', 'Болор', 'Болд']
index()	Заасан элементийн индексийг буцаана.	jims = ['алим','тоор','чангааНз'] y=jims.index("тоор") print('Y=',y)	Y=1

Жагсаалт

Функц	Тайлбар	Жишээ	Үр дүн
insert()	Заасан байрлалд элемент нэмнэ	jims = ['алим','тоор','чангааНз'] jims.insert(0,"тоор") print('Result:',jims)	Result: ['тоор', 'алим', 'тоор', 'чангааНз']
remove()	Заасан байрлалаас элемент устгана	jims = ['алим','тоор','чангааНз'] jims.remove("тоор") print('Result:',jims)	Result: ['алим', 'чангааНз']
reverse()	Жагсаалтын элементүүдийг урвуу дарааллаар байрлуулна	jims = ['алим','тоор','чангааНз'] name=['Туяа','Болор','Болд'] jims.extend(name) print('Result1:',jims) jims.reverse() print('Result2:',jims)	Result1: ['алим', 'тоор', 'чангааНз', 'Туяа', 'Болор', 'Болд'] Result2: ['Болд', 'Болор', 'Туяа', 'чангааНз', 'тоор', 'алим']
sort()	Жагсаалтыг эрэм-бэлнэ	jims = ['чангааНз','тоор','алим'] jims.sort() print('Result:',jims)	Result: ['алим', 'тоор', 'чангааНз']

Tuple

- Tuple- ялгаатай эсвэл ижил төрлийн элементүүдийг агуулсан дараалал юм.

Онцлог нь:

- Анхны үүсгэсэн дарааллын элементүүдийн утгыг өөрчлөх боломжгүй (immutable шинжтэй);
- Эрэмбэлэгдсэн: tuple нь зүйл бүрийн индексийн утгыг хадгалдаг ба энэ нь өгөгдөл оруулах дарааллыг агуулна;
- Өөрчлөгдөх боломжгүй: tuple-д зүйл нэмэх, устгах боломжгүй;
- Нэгэн төрлийн бус: tuple дэх ялгаатай өгөгдлийн дараалалд тэдгээрийг индексжүүлэх, зүсэх замаар хандана;
- Давхардсан зүйлсийг агуулсан: tuple нь давхардсан байж болох ба тэдгээр нь ижил утгатай зүйлстэй байж болно;
- Tuple-г солих үйлдэл хийхэд (x,y)=(y,x) гэж бичиж ашиглаж болно;
- Tuple-н өөр нэг онцлог хэрэглээ нь функцээс нэгээс илүү утга буцаах боломжтой.

Tuple

Жишээ: tuple-с “ >=10” нөхцөлийг хангах элементүүдийн тоог олох

```
col=0
tup=(13, 2, 23, 14, 5)
for i in tup:
    print(i, end=" ")
for i in range(5):
    if tup[i]>=10:
        col=col+1
print("\nTup-н >=10 нөхцөлийг хангах элементийн тоо:", col)
```

Үр дүн:

```
13 2 23 14 5
Tup-н >=10 нөхцөлийг хангах элементийн тоо: 3
```

Толь бичиг

Элементүүдийг тоон индексээр бус, дурын объектоор (тоо, тэмдэгт мөр, tuple) тодорхойлох боломжийг олгодог өгөгдлийн бүтцийг Python хэлний **толь бичиг** гэж нэрлэдэг.

Тодорхой дурын объект нь толь бичигт агуулагдсан объектуудын багц руу нэвтрэх боломжийг нээж өгдөг түлхүүр юм.

Толь бичиг (dictionary) нь түлхүүр (key) болон утга (value) хоорондын хамаарлын өөрчлөгдөх боломжтой, индекслэгдсэн олонлог юм.

Түлхүүр цор ганц байх ёстой. Толь бичгийг **{}** хаалтанд дүрслэх бөгөөд элемент бүр нь **key:value** гэсэн хослолоор дүрслэгдэнэ.

түлхүүр нь тэмдэгт мөр биш гэдгийг анхаарах.

Толь бичгийн бичиглэлийн дүрэм:

{key1:value1, key2:value2, keyN:valueN}

Жишээ:

```
dict={‘word’:’үг’,’sentence’:’өгүүлбэр’,’poem’:’шүлэг’}
print(dict)
```

Үр дүн:

{'word': 'үг', 'sentence': 'өгүүлбэр', 'poem': 'шүлэг'}

Толь бичиг

Толь бичиг дээр гүйцэтгэх үйлдлүүд болон ашиглах боломжтой функцууд

Толь бичигт хийх үйлдлүүд	**Тайлбар**	**Жишээ**	**Үр дүн**
Түлхүүрээр элементэд хандах	Харгалзах түлхүүрийг [ ] хаалтын хамт ашиглана.	dict={'word':'үг','sentence':'өгүүлбэр','poem':'шүлэг'} print('dict[word]':, dict['word'])	dict[word]: үг
Шинэ элемент нэмэх	Тухайн түлхүүрт оноож өгөх утгыг [ ] хаалтанд оруулнаар толь бичигт шинэ утга нэмнэ	dict['novel'] = 'роман'	{'word':'үг','sentence':'өгүүлбэр','poem':'шүлэг', 'novel': 'роман'}
Түлхүүрийн үгийг солих	Түлхүүртэй холбоотой утгыг [ ] хаалт ашиглан сольж болно.	dict['novel'] = 'өгүүлбэр' print(dict)	{'word':'үг','sentence':'өгүүлбэр','poem':'шүлэг', 'novel': 'өгүүлбэр'}
Элемент устгах pop(), popitem() функцууд, del түлхүүр үг, clear() функц ашиглана.	Толь бичгийн элемент устгагдах боломжтой. pop(key)- key түлхүүртэй элементийг устгаад харгалзах утгыг буцаана. popitem() -толь бичигт хамгийн сүүлд оруулсан элементийг устгана. del key-харгалзах түлхүүртэй элементийг устгаад утга буцаахгүй. clear() функц толь бичгийн бүх элементийг устгана.	dict={'word':'үг','sentence':'өгүүлбэр', 'poem':'шүлэг', 'novel': 'роман'} print(dict) dict.popitem() dict.pop('poem') print(dict) del dict['sentence'] print(dict) dict.clear() print(dict)	{'word':'үг','sentence':'өгүүлбэр', 'poem':'шүлэг', 'novel': 'роман'} {'word':'үг','sentence':'өгүүлбэр', 'poem':'шүлэг'} {'word':'үг','sentence':'өгүүлбэр' } {'word':'үг' } {}

Толь бичиг

Толь бичиг дээр гүйцэтгэх үйлдлүүд болон ашиглах боломжтой функцууд

Толь бичигт хийх үйлдлүүд	**Тайлбар**	**Жишээ**	**Үр дүн**
Толь бичгийн элементийг шинэчлэх	update() - толь бичгийн сүүлийн элементийг шинэчлэх	dict.update({‘picture book’:‘зураг ном’}) print(dict)	{‘word’:’үг’,’sentence’:’өгүүлбэр’, ’poem’:’шүлэг’, ‘novel’: ‘роман’,‘picture book’:‘зураг ном’}
Толь бичгийн агуулгыг харах	values()-толь бичгийн утгуудын агуулгын утга буцаана, keys()-толь бичгийн түлхүүрүүдийн харагдах байдлыг буцаана, items()-элементүүдийн утгыг (key, value) хослолоор буцаана	print(dict.values()) print(dict.keys()) print(dict.items())	[’үг’,’өгүүлбэр’, ’шүлэг’, ‘роман’,‘зураг ном’] [‘word’,’ sentence’, ’poem’, ‘novel’,‘picture book’] [(‘word’,’үг’),(’sentence’,’өгүүлбэр’),(’poem’:’шүлэг’),(‘novel’,‘роман’),(‘picture book’,‘зураг ном’)]
Толь бичгийн хуулбарыг буцаах	copy() method ашглана. Энэ нь аргументгүй бичигддэг. dictionary.copy()	d = {1:'one',2:'two',3:'three'} d_copy = d.copy() d_copy[4] = 'four' d_copy[5] = 'five' d_copy[6] = 'six' print(d) print(d_copy)	{1: 'one', 2: 'two', 3: 'three'} {1: 'one', 2: 'two', 3: 'three', 4: 'four', 5: 'five', 6: 'six'}

Толь бичиг

Толь бичиг дээр гүйцэтгэх үйлдлүүд болон ашиглах боломжтой функцууд

Толь бичигт хийх үйлдлүүд	**Тайлбар**	**Жишээ**	**Үр дүн**
Тодорхойлсон түлхүүр ба утгыг агуулсан толь бичгийг буцаана	fromkeys() method ашглана dict.fromkeys(keys,value)	keylist = ('A', 'B', 'C', 'D') value = 0 d = dict.fromkeys(keylist, value) print(d)	{'A': 0, 'B': 0, 'C': 0, 'D': 0}
Заасан түлхүүрийн харгалзах утгыг буцаана	get() method ашглана dictionary.get(key, value)	d = {1:'one', 2:'two', 3:'three'} print(d.get(3)) print(d.get(4)) print(d.get(4, 'empty'))	three None empty
Заасан түлхүүрийн харгалзах утгыг засварлана	setdefault() method ашглана dictionary.setdefault(key, value)	d = {1:'one', 2:'two', 3:'three'} print(d.setdefault(3)) print(d.setdefault(4, 'four')) print(d.setdefault(5)) print(d)	three four None {1: 'one', 2: 'two', 3: 'three', 4 'four', 5: None}
Толь бичигт объект байгаа эсэхийг шалгана	in түлхүүр үг ашиглана	dict={‘word’:’үг’,’sentence’:’өгүүлбэр’, ’poem’:’шүлэг’, ‘novel’: ‘роман’} print(‘word’ in dict) print(‘үг’ in dict)	True False

Олонлог

Set-Олонлог гэдэг нь ямар ч давхардалгүй, эрэмбэлэгдээгүй элементүүдийн багц буюу цуглуулга юм. Олонлогууд нь элементүүдийн багц хоорондын нэгдэл, огтлолцол, ялгааг олох зэрэг логик үйлдлүүдийг хийхэд тохиромжтой.
Жишээлбэл: олон нийтийн сүлжээн дэх найзуудаа тодорхойлоход олонлог ашиглаж болно.
Эрэмбэлэгдээгүй элементүүдтэй тул олонлогийн элементэд хандах боломжгүй.
Олонлогт **{ }** хаалт ашиглана.

Олонлогийн бичиглэлийн дүрэм:

```
set_A = set(iterable)
set_B = {element_A, element_B, ..., elementZ}
```

Жишээ: Олонлог үүсгэх ба хэвлэх

```
amitad = set()
print(amitad)
amitad = set(['mogoi', 'mazaalai', 'irves'])
nuur= {'Khuvsgul', 'Uvs', 'Buir', 'Khar us', 'Tolbo'}
nohoi = {'pug', 'golden retriever', 'corgi', 'pug', 'corgi'}
print("Amitadiin_olonlog:", amitad)
print("Nuuruud:", nuur)
print("Nohoin_uuldruud:", nohoi)
```

Үр дүн:

```
set()
Amitadiin_olonlog : {'irves', 'mazaalai', 'mogoi'}

Nuuruud: {'Tolbo', 'Khar us', 'Buir', 'Uvs',
'Khuvsgul'}

Nohoin_uuldruud: {'corgi', 'golden retriever',
'pug'}
```

Олонлог

Олонлогт ашиглах гишүүн функцууд ба үйлдлүүд

Функц ба үйлдэл	Тайлбар	Жишээ	Үр дүн
.add()	Олонлогт элемент нэмэх set.add(element) хоосон олонлог үүсгэхэд () хаалт ашиглана.	numbers = set() numbers.add(1) numbers.add(2) print(numbers)	{1,2}
.clear()	Олонлогийн бүх элемент устгана set_object.clear()	amitad = set(['mogoi', 'mazaalai', 'irves']) print("Amitadiin_olonlog:", amitad) amitad.clear() print(amitad)	Amitadiin_olonlog : {'irves', 'mazaalai', 'mogoi'} clear() фнкцийн дараа: set()
.copy()	Олонлогийн элементүүдийн хуулбарыг буцаана set.copy()	original_set = set(['mogoi', 'mazaalai', 'irves']) print("Amitadiin_olonlog:", amitad) new_set=amitad.copy() new_set.add('alim') print(new_set)	Amitadiin_olonlog : {'irves', 'mazaalai', 'mogoi'} {'irves', 'mazaalai', 'mogoi','alim'}
.difference()	Хоёр болон түүнээс дээш олонлогийн зөрүүг агуулсан олонлогийг буцаана set.difference(set1, set2, ..., setN) setA - setB - setC **нь - ижил**	bat_durtai= set(["gadil", "alim", "chavga", "liir"]) dorj_durtai = set(["gadil", "alim", "chavga", "uzem"]) print(bat_durtai-dorj_durtai)	{'liir'} энэ нь зөвхөн эхний олонлогт хамаарах үр дүнг хэвлэв

Олонлог

Олонлогт ашиглах гишүүн функцууд ба үйлдлүүд

Функц ба үйлдэл	**Тайлбар**	**Жишээ**	**Үр дүн**
.difference _update()	Энэ олонлог дахь өөр олонлогт агуулагдсан элементүүдийг устгана set.difference_update(set1, set2, ..., setN) нь **- = ижил**	salat = {'cezar', 'niislel', 'vinegret', 'holimog', 'tahianii'} zuush= {'cezar', mahan', 'holimog', 'cheese', 'tahianii'} salat.difference_update(zuush) print(salat)	{'niislel','vinegret'}
.discard(value)	Заасан элементийг устгана set.discard(value)	salat = {'cezar', 'niislel', 'vinegret', 'holimog', 'tahianii'} print(salat) salat.discard('vinegret') print(salat)	{'cezar', 'niislel', 'vinegret', 'holimog', 'tahianii'} {'cezar', 'niislel', 'holimog', 'tahianii'}
1.intersection() 2.intersection _update()	Огтлолцол гэдэг нь хоёр олонлогийн аль алинд байдаг утгууд юм. 1.Хоёр өөр олонлогийн огтлолцсон олонлогийг буцаана. set.intersection(set1, set2, ..., setN) setA & setB & setC нь **& ижил** 2.Энэ олонлогт байгаа бусдад байхгүй элементүүдийг устгана. set.intersection_update(set1, set2 ... etc) нь **&= ижил**	univ_gr={'Muugii','Enkh','Nasaa','Aagii'} off_gr={'Zoloo','Nasaa','Enkh','Duka'} mutual_gr=univ_gr.intersection(off_gr) print(mutual_gr)	{'Nasaa','Enkh'}
.isdisjoint()	Хоёр олонлог огтлолцож байгаа эсэхийг буцаана. set1.isdisjoint(set2)	t1 = {10, 20, 30} t2 = {40, 50, 60} t3 = {30, 40, 50} print(t1.isdisjoint(t2)) print(t1.isdisjoint(t3))	True False

Олонлог

Олонлогт ашиглах гишүүн функцууд ба үйлдлүүд

Функц ба үйлдэл	Тайлбар	Жишээ	Үр дүн
.issubset()	Нэг олонлогийг нөгөө олонлог агуулж байгаа эсэхийг шалгаж Үнэн эсвэл Худал утга буцаана. set.issubset(set1) нь set <= set1 **ижил**	t1 = {10, 20, 30} t3 = {30, 40, 50} print(t1.issubsett(t3))	True
.pop() .remove(item)	Олонлогийн сүүлчийн элементийг устгана set.pop() Олонлогоос заасан элементийг устгана set.remove(item)	t1 = {10, 20, 30} t1.pop() print(t1) t1 = {10, 20, 30} t1.remove(20) print(t1)	{10, 20} {10,30}
.symmetric _difference(set1) .symmetric _difference _update(set1)	Хоёр олонлогийн тэгш хэмийн зөрүүтэй олонлогийг буцаана set.symmetric_difference(set1) Тухайн олонлогоос бусад олонлогт тэгш хэмтэйгээр ялгааг оруулах set.symmetric_difference_update(set1) нь **^= ижил**	t1 = {10, 20, 30} t3 = {50, 40, 10} t=t1.symmetric_difference(t2) print(t) t1 = {10, 20, 30} t3 = {50, 40, 10} t1.symmetric_difference_update(t2) эсвэл **t1^= t3** print(t1)	{20, 30, 50, 40} {50, 30,20,40}
.union(set1, set2...) .update(set1, set2 ...)	Олонлогуудын нэгдлийг агуулсан багцыгбуцаана. set.union(set1, set2...) нь **set | set1 | set2 ... ижил** Олонлогуудын нэгдлийг шинэчлэх set.update(set1, set2 ...) **set |=set1 |= set2 ... ижил**	x = {"a", "b", "c"} y = {"f", "d", "a"} z = {"c", "d", "e"} result = x.union(y, z) эсвэл **result= x|y|z** print(result) x = {"a", "b", "c"} y = {"f", "d", "a"} x|=y **эсвэл x.update(y)** print(result)	{'d', 'f', 'c', 'e', 'b', 'a'} {'a','b','c','f','d'}

Функц

Зарим үйлдлийг программын код дотор олон удаа гүйцэтгэх шаардлагатай. Нэг кодыг олон газар дахин бичихийн оронд **def** түлхүүр үг ашиглан функцийг тодорхойлж болно. Өөрөөр хэлбэл зөвхөн дуудсан үед л ажилладаг блок кодыг **Функц** гэнэ.

Параметр гэж нэрлэгддэг өгөгдлийг функц руу дамжуулж, функцийн биед тодорхойлсон үйлдлийг гүйцэтгээд үр дүнгийн утгыг функц буцаах боломжтой. **Функц** нь **нэртэй**, **параметртэй**, **утга буцаадаг**, **буцаадаггүй**.

Python хэлэнд стандарт функцууд, хэрэглэгчийн тодорхойлсон, урьдчилан суулгасан модулиудад агуулагдах функцууд (method), Lambda, Filter, Map, Reduce функцууд ашигладаг.

Хэрэглэгчийн функц тодорхойлох дүрэм:

```
def функцийн нэр(параметрүүд):
        функцийн бие
        return буцаах утга
```

Жишээ: Параметрын утгыг 10 дахин нэмэгдүүлэн буцаах функц бичнэ үү.

```
def my_function(x):
  return 10 * x
print(my_function(6))
print(my_function(7))
print(my_function(8))
```

Үр дүн:

```
60
70
80
```

Функц-Lambda

Lambda функц нь нэргүй жижиг функц юм. **Lambda функц** нь хэдэн ч аргумент авч болох боловч зөвхөн нэг илэрхийлэлтэй байж болно. **Lambda функц** нь **def** түлхүүр үгний дүрмийг ашиглахгүйгээр тодорхойлогдоно.

Lambda функц нь Filter, Map, Reduce функцуудтэй хоршиж ажилладаг.

Lambda функцийн бичиглэл:

lambda *arguments* : *expression*

Lambda нь функцийн түлхүүр үг

Lambda функцэд argument – аргумент нь хэд ч байж болох ба харин буцаах илэрхийлэл нэг байна.

Мөн утга буцаагүй байж болно

Жишээ: S аргументын утгыг 100-р нэмэгдүүлээд үр дүнг буцаахад Lambda функц ашиглана уу.

```
z = lambda S: S + 100
print(z(50))
```

Үр дүн:
150

```
def z(s):
        return s+100
print(z(50))
```

Үр дүн:
150

Функц-Filter

Filter() функц нь элементийг хүлээн зөвшөөрсөн эсэхийг шалгах функцийг ашиглан шүүж байгаа давталтыг буцаана.

filter функцийн бичиглэл:

filter(*function*, *iterable*)

***function*–** Давталт дахь элемент бүрийн хувьд ажиллах функц.

***Iterable*-** шүүгдэх боломжтой давталтын объект

Жишээ: Массивыг шүүж, зөвхөн 18-тай тэнцүү буюу түүнээс дээш утгатай шинэ массив буцаахад filter() функц ашиглана уу.

```
nas = [3,11,16,18,25,35]

def myFunc(n):
    if n<18:
        return False
    else:
        return True

nasand_huregch=filter(myFunc, nas)

for n in nasand_huregch:
    print(n)
```

Үр дүн:

18
25
35

Add 10 to argument a, and return the result.

Функц-Map

Map функц нь давталт дахь элемент бүрийн хувьд заасан функцийг гүйцэтгэдэг.

Элемент нь функцэд параметр болж илгээгддэг.

map функцийн бичиглэл:

map(*function*, *iterables*)

***function*–**элемент бүрийн хувьд гүйцэтгэх функц.

Iterable- Дараалал, давтагдах объект. Давтагдах элементийн тоо хэд ч байж болно. Энэ функц нь давтагдах объект бүрийн хувьд нэг параметртэй байна.

Жишээ: дахь үг бүрийн уртыг тооцоолоход map функц ашиглана уу.

```
def myfunc(s):
  return len(s)

s = map(myfunc,('aarts', 'tsetseg', 'urum'))
print(s)

print(list(s))
```

Үр дүн:
[5,7,4]

Функц-Reduce

Reduce функц нь функцийг давтагдах элементэд ашиглах бөгөөд элемент тус бүрийн буцсан үр дүнг нэгтгэн нэг үр дүн болгон авдаг.

Reduce() функц нь functools модулийн нэг хэсэг тул Reduce() функцийг ашиглахаасаа өмнө импортлох шаардлагатай.

Reduce функцийн бичиглэл:

functools.reduce(function, iterable[, initializer])

***function*–**функц нь 2 аргумент авдаг ба тэдгээр дээр үйлдэл гүйцэтгэдэг.

Iterable- Элементүүд нь функцээр боловсруулагдан давтагдах боломжтой.

Initializer-хувьсагчийн анхны утгыг энд олгож болно.

Жишээ: 1-5 хүртэлх тооны нийлбэр олоход reduce функц ашиглана уу.

```
from functools import reduce

#Хоёр тоо нэмэх функц
def add(x, y):
    return x + y

a = [1, 2, 3, 4, 5]
res = reduce(add, a)

print(res)
#Output: 15
```

Үр дүн:
15

Дүгнэлт

Энэ удаагийн лекцээр нийлмэл өгөгдлийн төрлүүд болох

- Массив ба жагсаалт;
- Tuple, Толь бичиг, олонлогийн талаарх ойлголтууд ба хэрэглээг;
- Lamda, Filter, Map, Reduce функцуудыг хэрхэн хэрэглэх болон хэрэглэгчийн функц хэрхэн бичих, дуудах талаарх ойлголтуудыг өгсөн болно.

Эдгээр ойлголтуудыг лабораторийн хичээлээр туршиж үзээд бие дааж бодлого бодох болно.

Анхаарал тавьсанд баярлалаа.

Асуултаа сургалт удирдах системийн зурвасаар илгээнэ үү.